

Sure — here is **only the complete code**, with the GitHub structure and `requirements.txt` removed.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# 1. Load Dataset
df = pd.read_csv("house_price_dataset.csv")

# 2. Clean Column Names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# 3. Remove Duplicates
df = df.drop_duplicates()

# 4. Convert Numerical Columns
for column in [
    "totalprice",
    "sqft",
    "bhk",
    "pricepersqft"
]:
```

```
df[column] = pd.to_numeric(
    df[column],
    errors="coerce"
)

# 5. Remove Missing Target Values
df = df.dropna(
    subset=["totalprice"]
)

print(df.head())
print("Dataset Shape:", df.shape)

# 6. Exploratory Data Analysis
plt.figure(figsize=(8, 5))

sns.histplot(
    df["totalprice"],
    kde=True
)

plt.title("Distribution of House Prices")
plt.xlabel("Price")
plt.show()

# 7. Area vs Price
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="sqft",
    y="totalprice"
)

plt.title("Area vs House Price")
plt.xlabel("Area (sqft)")
plt.ylabel("Price")
plt.show()

# 8. Features and Target
X = df[
```

```

        [
            "bhk",
            "propertytype",
            "location",
            "sqft",
            "pricepersqft"
        ]
    ]

y = df["totalprice"]

# 9. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# 10. Preprocessing
numeric = [
    "bhk",
    "sqft",
    "pricepersqft"
]

categorical = [
    "propertytype",
    "location"
]

preprocessor = ColumnTransformer([
    (
        "num",
        SimpleImputer(strategy="median"),
        numeric
    ),
    (
        "cat",
        Pipeline([
            (

```

```

        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
]),
categorical
)
])

```

11. Random Forest Model

```

model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "regressor",
        RandomForestRegressor(
            n_estimators=100,
            random_state=42
        )
    )
])

```

12. Train Model

```

model.fit(
    X_train,
    y_train
)

```

13. Prediction

```

y_pred = model.predict(
    X_test
)

```

```

# 14. Model Evaluation
mae = mean_absolute_error(
    y_test,
    y_pred
)

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)

r2 = r2_score(
    y_test,
    y_pred
)

print("\nModel Evaluation")
print("-----")
print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# 15. New House Prediction
new_house = pd.DataFrame({
    "bhk": [2],
    "propertytype": ["Apartment"],
    "location": ["Pune"],
    "sqft": [1000],
    "pricepersqft": [5000]
})

prediction = model.predict(
    new_house
)

print(
    "\nPredicted House Price: ₹",

```

```
prediction[0]  
)
```